PARALLEL COMPUTING PERFORMANCE // LAB HANDOUT

Empirical Scalability Diagnostics: The Karp–Flatt Metric

Distinguishing Algorithmic Serialization from Inter-Core Contention in Python

INSTRUCTOR	DURATION	ASSESSMENT	STACK & DELIVERY
Sufyan bin Uzayr	120 Minutes (Solo)	Ungraded Practice Lab	Python 3.8+ / Git Branch

THE ARCHITECTURAL DIAGNOSTIC PRINCIPLE

When multi-core software yields sublinear speedup, you are suffering from one of two orthogonal failure modes: an unparallelized serial fraction (Amdahl) or resource contention over shared memory and locks. The Karp–Flatt metric separates these two bottlenecks empirically:

Speedup: $S_p = T_1 / T_p$ **Karp–Flatt:** $e(p) = [(p \cdot T_p / T_1) - 1] / (p - 1)$ (for $p \geq 2$)

Diagnostic Decision Rule: If $e(p)$ stays constant across worker counts, you have a fixed serial code path. If $e(p)$ grows monotonically with core count, your threads are thrashing over locks, cache lines, or bus bandwidth.

Practice Lab Schedule

120 MINUTES TOTAL

Timeline	Target Phase	Key Activity	Git Artifact
00:00 – 00:15 (15 min)	Part 0: Workspace Setup	Repo clone, CPU topology check, branch creation	`practice-<ID>`
00:15 – 00:50 (35 min)	Part 1: Experiment A	Measure stubborn serial tail; observe flat $e(p)$	`exp_a_serial.py`
00:50 – 01:25 (35 min)	Part 2: Experiment B	Reproduce lock contention traffic jam; observe $e(p)$ surge	`exp_b_contention.py`
01:25 – 01:50 (25 min)	Part 3: Experiment C	Implement thread-local privatization; restore scaling	`exp_c_fixed.py`
01:50 – 02:00 (10 min)	Part 4: Final Submission	Document metrics in `RESULTS.md`, push to Git	`git push origin`

Part 0: Workspace & Branch Setup

15 MINUTES

Clone the practice repository, create your working branch, and verify logical cores available on your machine:

```
# 1. Clone repository and create student branch git clone <ASSIGNED_REPO_URL> lab-karp-flatt && cd lab-karp-flatt git checkout -b practice-<YOUR_STUDENT_ID> # 2. Inspect available execution cores on host machine python3 -c "import os; print(f'Execution Cores: {os.cpu_count()}')"
```

Part 1: Experiment A — The Stubborn Serial Tail

35 MINUTES

In this experiment, compute work is parallelized cleanly across worker processes, but the master process executes an intentional 0.40s sequential pause before dispatching tasks. Implement `exp_a_serial.py`:

```
import time, math from multiprocessing import Pool def heavy_math(n): return sum(math.sin(i) * math.cos(i) for i in range(1, n)) def run_experiment(workers, total_chunks=16, chunk_size=2_000_000): t0 = time.perf_counter() time.sleep(0.40) # Intentional sequential bottleneck (~12% serial fraction) with Pool(processes=workers) as pool: pool.map(heavy_math, [chunk_size] * total_chunks) return time.perf_counter() - t0
```

Task: Benchmark across $P \in \{1, 2, 4\}$ cores. Compute S_p and $e(p)$. Verify that $e(2)$ and $e(4)$ hover closely around ~0.10–0.14, confirming that performance loss is purely algorithmic. Commit as `"Part 1: Experiment A complete"`.

Part 2: Experiment B — The Contention Traffic Jam

35 MINUTES

Now we eliminate the artificial sleep timer. The algorithm is theoretically 100% parallelizable. However, we force all worker processes to increment a single shared integer protected by a mutual exclusion lock on every single iteration.

THE PHYSICAL REALITY OF LOCK CONTENTION (SLIDE 44 REVIEW)

Lock-free does not mean contention-free, and lock acquisition is not free. When multiple cores compete for a shared mutex, execution pipelines stall in kernel sleep states or burn cycles in spinloops. Crucially, the cache line holding the lock and counter bounces rapidly across the CPU interconnect. As you add cores, coordination overhead dominates arithmetic, creating **negative speedup** ($S_p < 1$) and an exploding Karp–Flatt score.

Create and implement `exp_b_contention.py` using Python's native multiprocessing primitives:

```
import time from multiprocessing import Process, Value, Lock def contended_worker(shared_counter, lock, iterations): for _ in range(iterations): with lock: # Heavy lock contention & cache-line bouncing shared_counter.value += 1 def run_contended(p, total_increments=2_000_000): increments_per_worker = total_increments // p counter = Value('i', 0) lock = Lock() procs = [ Process(target=contended_worker, args=(counter, lock, increments_per_worker)) for _ in range(p) ] t0 = time.perf_counter() for proc in procs: proc.start() for proc in procs: proc.join() t1 = time.perf_counter() assert counter.value == total_increments return t1 - t0 if __name__ == "__main__": cores = [1, 2, 4] runtimes = {p: run_contended(p) for p in cores} t1 = runtimes[1] for p in [2, 4]: tp = runtimes[p] sp = t1 / tp e = ((p * tp / t1) - 1.0) / (p - 1.0) print(f"P={p} | Runtime={tp:.3f}s | Speedup={sp:.2f}x | Karp-Flatt e={e:.3f}")
```

Empirical Observations to Record:

- **Negative Speedup:** Notice how T_4 is substantially larger than T_1. Adding cores made the application run significantly slower.
- **Metric Surge:** Calculate $e(2)$ and $e(4)$. Unlike Experiment A, the Karp–Flatt value will not remain flat—it will surge past 0.50 and often exceed 1.50.
- **Contention Signature:** This divergence proves mathematically that the slowdown is not due to sequential algorithm code, but rather catastrophic cross-thread contention.

EXPECTED DIAGNOSTIC SIGNATURE COMPARISON

Scenario	T_1	T_2	T_4	$e(2)$	$e(4)$	Karp–Flatt Diagnosis
Exp A (Serial Tail)	3.20s	1.80s	1.10s	**0.125**	**0.120**	Flat e → Fixed Amdahl serial path (sleep)
Exp B (Contention)	0.65s	1.10s	2.40s	**1.380**	**3.920**	Surging e → Lock thrashing & coherence traffic

GIT CHECKPOINT 2

Commit your working script to your local branch:

`git add exp_b_contention.py && git commit -m "Part 2: Captured lock contention pathology and Karp-Flatt surge"`

Part 3: Experiment C — The Privatization Fix

25 MINUTES

To eliminate lock contention, apply **Golden Rule 8: Privatize before synchronizing**. Each worker process accumulates increments into a private local CPU register/stack variable with zero locks. Only upon loop termination do the workers combine their results via a single reduction.

Implement the privatized counter pattern in `exp_c_fixed.py`:

```
import time from multiprocessing import Pool def private_worker(iterations): # Thread-local accumulation in private memory space (zero shared state) local_sum = 0 for _ in range(iterations): local_sum += 1 return local_sum def run_privatized(p, total_increments=20_000_000): increments_per_worker = total_increments // p tasks = [increments_per_worker] * p t0 = time.perf_counter() with Pool(processes=p) as pool: results = pool.map(private_worker, tasks) total = sum(results) # Synchronize exactly ONCE at the end t1 = time.perf_counter() assert total == total_increments return t1 - t0
```

Task: Run `exp_c_fixed.py` for $P \in \{1, 2, 4\}$. Note that 20,000,000 increments now execute in fractions of a second. Observe near-linear speedup and a Karp–Flatt value that collapses back to $e \leq 0.03$.

Part 4: Results Synthesis & Git Push

10 MINUTES

Create a file named `RESULTS.md` in the root of your repository. Fill in your hardware profile and empirical runtimes across all three experiments:

Experiment Mode	T_1 (s)	T_2 (s)	T_4 (s)	$e(2)$	$e(4)$	Architectural Diagnosis
A: Serial Tail	*[Your val]*	*[Your val]*	*[Your val]*	~0.10–0.15	~0.10–0.15	Constant e → Fixed Amdahl serialization
B: Lock Contention	*[Your val]*	*[Your val]*	*[Your val]*	> 0.50	> 1.50	Exploding e → Mutex & bus thrashing
C: Privatized State	*[Your val]*	*[Your val]*	*[Your val]*	< 0.05	< 0.05	Collapsed e → Linear scalability restored

Finalize your git provenance and push your solution branch to the course remote:

```
git add exp_c_fixed.py RESULTS.md git commit -m "Part 3 & 4: Completed privatization fix and verified RESULTS.md" git push origin practice-<YOUR_STUDENT_ID>
```

INSTRUCTOR IN-LAB VERIFICATION RUBRIC (SUFYAN BIN UZAYR)

Raise your hand during the lab session for an in-person or remote Git sign-off. The instructor will verify:

- **Experiment A Check:** Ratio $e(4)$ / $e(2)$ is within [0.80, 1.25] (evidence of flat algorithmic serial fraction).
- **Experiment B Check:** Measured $T_4 > T_1$ and $e(4) >> e(2)$ (evidence of empirical lock thrashing).
- **Experiment C Check:** Measured $T_4 < T_1$ with parallel efficiency $E_4 > 70\%$ (successful state privatization).